FROM MODEL DIFFS TO CAUSAL MECHANISMS

Using a CrossCoder to explain behavioral differences between base and fine-tuned code models

The case

The model solved the task—then kept writing.

A single latent feature helped control that continuation.

RESEARCH QUESTION

The evaluation tells us what changed. The CrossCoder asks why.

BEHAVIOR

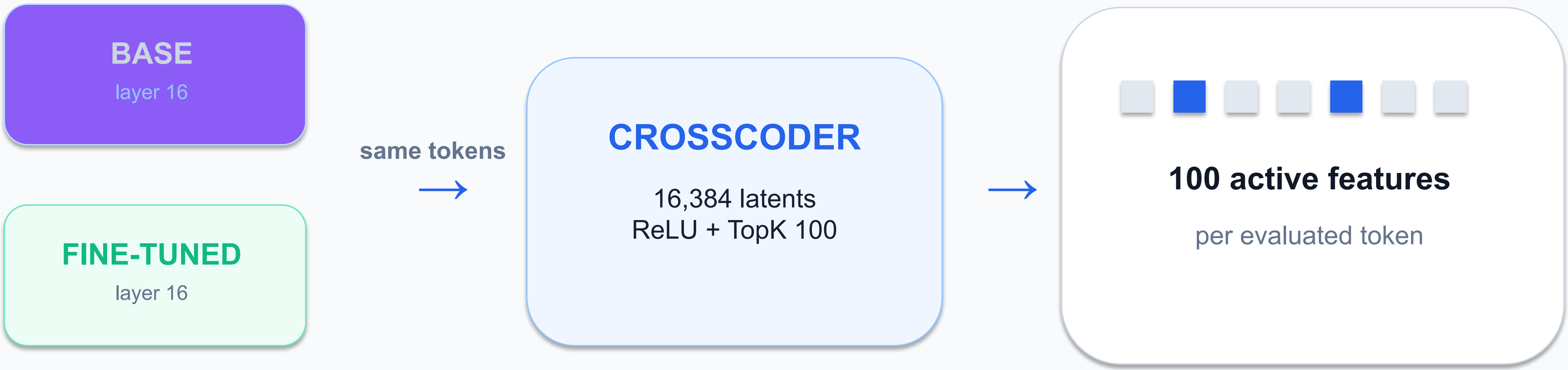
- Base passes / fine-tuned fails
- Base fails / fine-tuned passes
- What kind of error occurred?

REPRESENTATION

- Paired layer-16 residuals
- Shared sparse features
- Decoder directions for intervention

Behavior localizes the difference. Features turn it into a testable hypothesis.

First case study: DeepSeek base ↔ fine-tuned model diffing (DSTK100)



2,608 code texts · 617,959 evaluated tokens · same-text alignment

Method Lineage

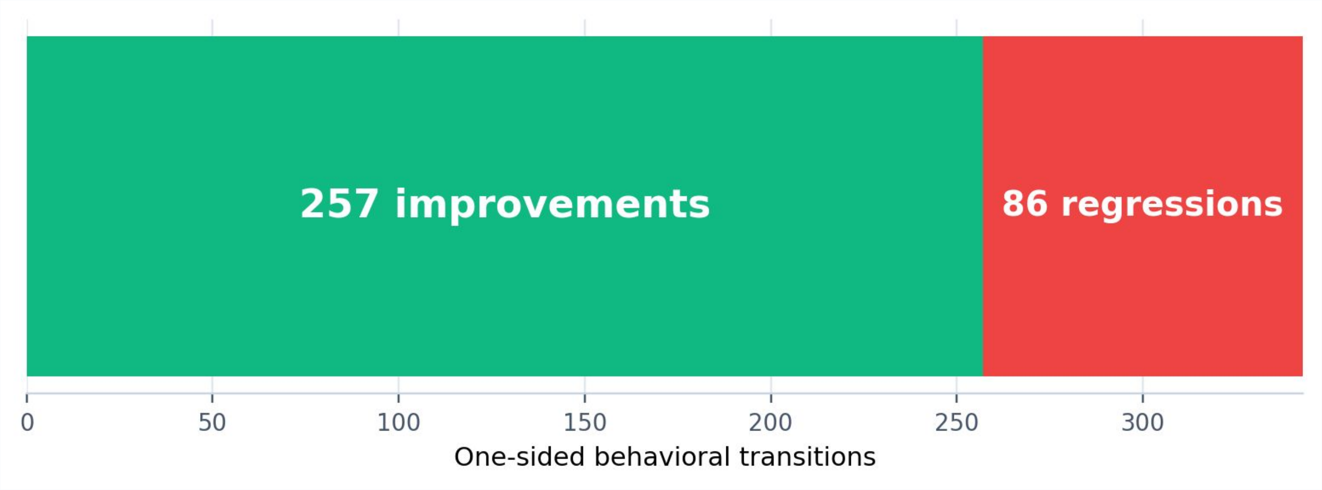
Based on *Sparse Crosscoders for Cross-Layer Features and Model Diffing* (Lindsey et al., 2024). TopK sparsity follows the sparse-autoencoder design studied by Gao et al. (2024). **DSTK100** is our same-text, layer-16, TopK-100 implementation.

References

- Lindsey, J. et al. "Sparse Crosscoders for Cross-Layer Features and Model Diffing." *Transformer Circuits*, 2024.
- Gao, L. et al. "Scaling and Evaluating Sparse Autoencoders." 2024.

BEHAVIORAL MAP

Ours Results to DSTK100 agrees with ReCatcher



343
One-Sided Transitions
Total instances where models diverged

257
Improvements
base fail → FT pass
HumanEval+: 42
BigCodeBench: 215

86
Regressions
base pass → FT fail
HumanEval+: 7
BigCodeBench: 79

Agreement with ReCatcher

ReCatcher reported that Kotlin fine-tuning improved aggregate General Logic for DeepSeek-Coder, despite introducing localized regressions—particularly syntax errors.

ReCatcher · DeepSeek FT vs original

- HumanEval+: **+14.94%** in Incorrect Code / General Logic
- BigCodeBench: **+9.32%** in Incorrect Code / General Logic

Our paired evaluation

- HumanEval+: **+22.57 pp** in pass rate
- BigCodeBench: **+7.19 pp** in pass rate
- Improvements outnumbered regressions by **~3:1** overall

* The magnitudes are not directly comparable: ReCatcher used 10 generations per prompt and its own regression-percentage definition; our study used one canonical generation per task and official paired evaluation.

Abbassi et al., ReCatcher: Towards LLMs Regression Testing for Code Generation, 2025.

FEATURE SCREENING

We ranked stable contrasts—not raw activation alone

4 summaries

max · early_max
mean · active fraction

→

Δ per task

aggregate(FT text)
− aggregate(base text)

→

Observed effect

mean Δ positives
− mean Δ controls

→

200 permutations

shuffle labels;
recompute effect

→

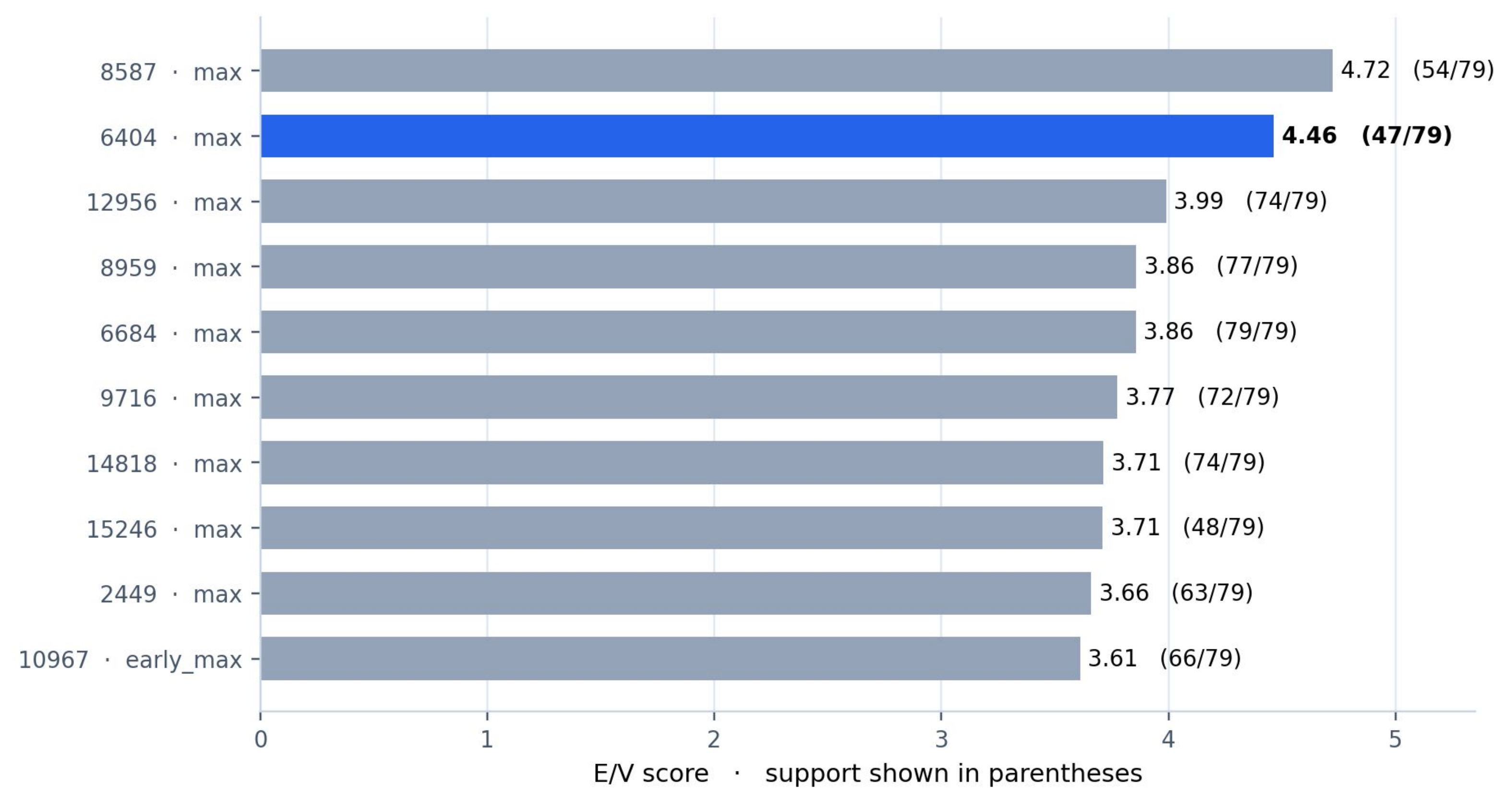
E/V

effect ÷ null SD

Minimum support **max(3 tasks, 10% of positives)**

E/V is a permutation signal-to-noise score—not a z-score or a p-value.

Feature 6404 ranked #2—and the result was specification-robust



6404

E/V 4.47

47 / 79 support

Same Top 10 with all four aggregations or E/V alone

A good causal candidate aligns meaning, failure mode, and timing

MEANING

What tokens and contexts
activate the feature?

FAILURE MODE

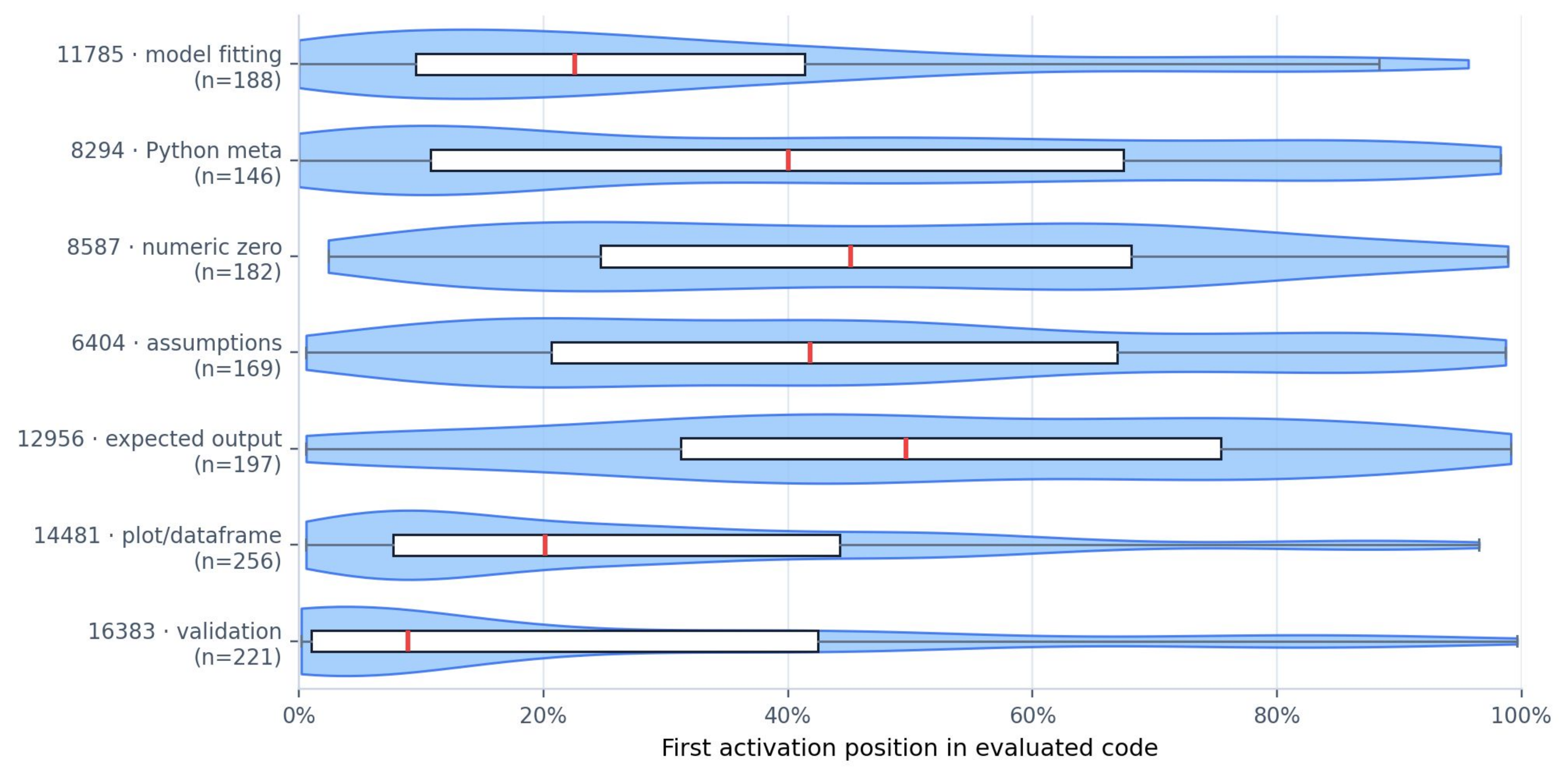
Which behavioral transition
is it associated with?

TIMING

Does activation precede the relevant decision?

Semantic clarity alone is not enough: a late feature may be a consequence or style marker.

First-activation position separates plausible causes from late



Example

Feature 12956

Expected output / examples

Semantically clear—often too late to explain the implementation decision.

The pearl: many ‘improvements’ were simply cleaner stopping behavior

119 / 215

BigCodeBench improvements

55%

test/import contamination in the base output

```
 def task_func(...):
     ... plausible solution ...
-
-# tests
-from task_func import task_func
```

The fine-tuned model often won by stopping—not by implementing a different algorithm.

Feature 6404 matched the semantics of post-solution continuation

6404

Dominant activation contexts

- expected
- should
- assumptions / caveats
- comments and boilerplate

67% vs **31%**

contamination cases | other improvements

Exploratory odds ratio ≈ 4.44

Association generated the hypothesis; it did not establish causality.

A deliberately selected cohort turned the hypothesis into a test

1
Base failed

2
Fine-tuned passed

3
Base generated tests/imports

4
Feature 6404 was naturally active

80 mechanistic cases

Selected for mechanism—not an out-of-sample estimate.

We intervened only when 6404 entered the natural TopK

Current token — capture layer 16 → **Paired residuals** — RMS normalize → **DSTK100** — ReLU + TopK 100 → **6404 active?** — gate intervention → **Next token** — continue generation

$$h'_{\square} = h_{\square} + \alpha \cdot z_{6404,\square} \cdot \mathrm{RMS}(h_{\square}) \cdot d_{6404,\mathrm{base}}$$

The intervention changes the next-token distribution; it does not rewrite earlier tokens.

TOKEN-LEVEL CASE

One example: the baseline solved the task, then imported its own test

BASELINE · FAIL

```
return []

# tests/test_task_func.py
import unittest
from task_func import task_func
```

The plausible function body is followed by test/import contamination.

6404 SUPPRESSION · PASS

```
return min_subsequence

# task_func({...})
# task_func({...})
```

Official pass; no generated test module import.

Red marks: online steps where feature 6404 entered the active TopK in the steered trajectory.

MAIN CAUSAL RESULT

The intervention recovered official passes—not just prettier text

19 / 80

fail → official pass

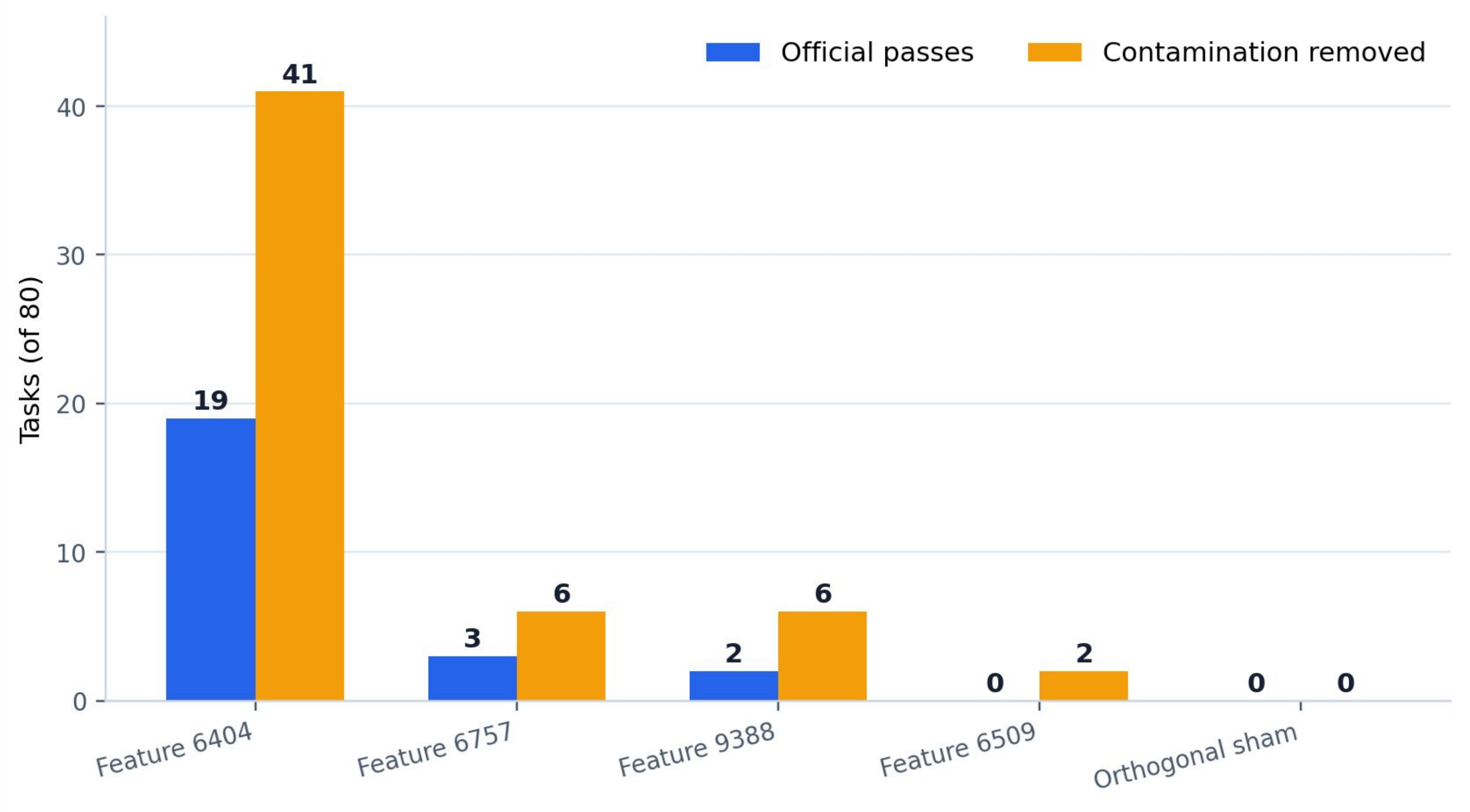
41 / 80

test/import contamination removed

$\alpha = -2$ · TopK-gated · exact code evaluated with extraction v4

Cohort selected for this mechanism; do not read 23.75% as general benchmark gain.

Matched controls isolate the 6404 direction



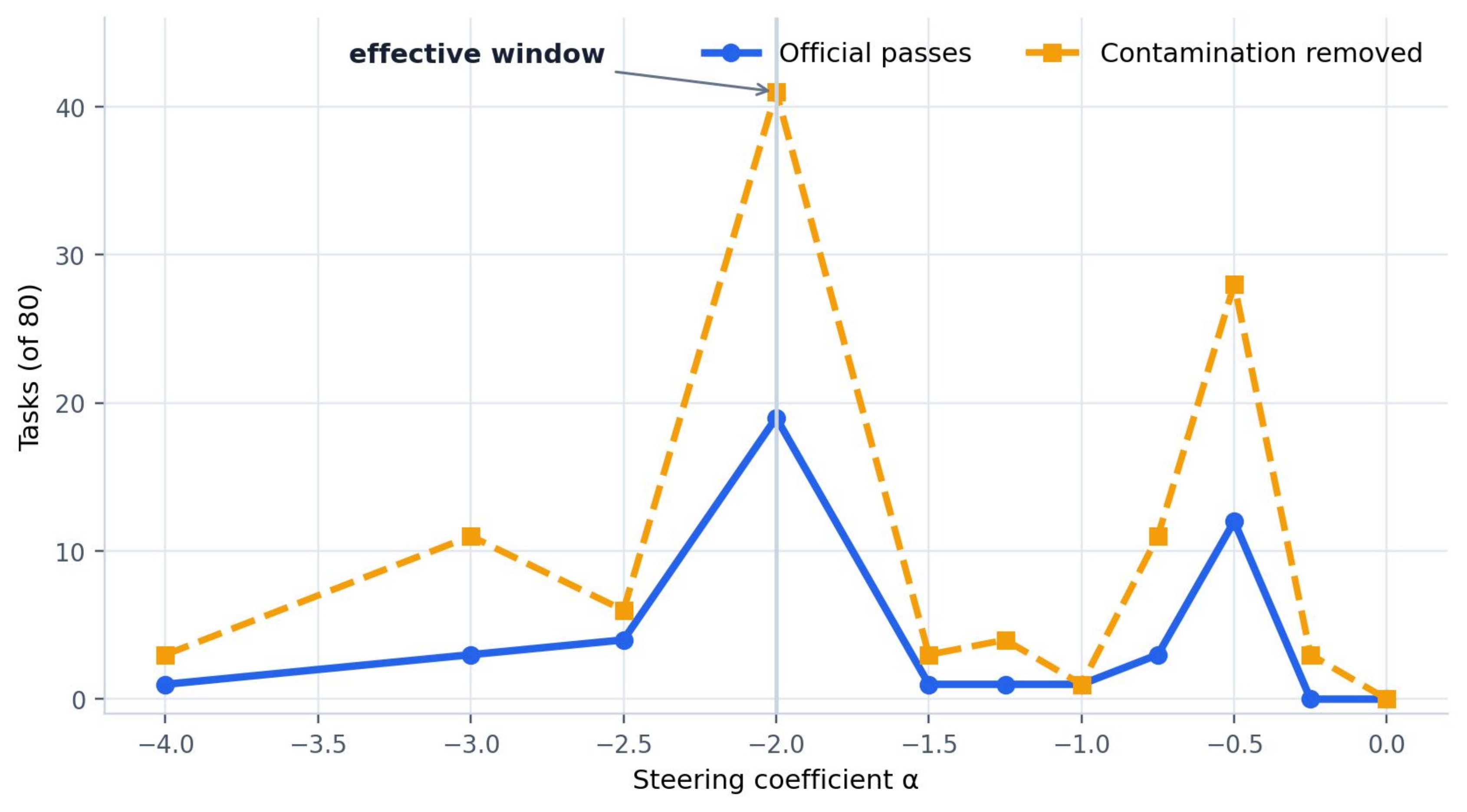
Controls matched

- decoder norm
- activation support
- temporal profile
- online gate activity

vs sham

pass $p \approx 3.8\times10^{-6}$
cleanup $p \approx 9.1\times10^{-13}$

Steering is not a volume knob



Why jagged?

- Token choice is discrete.
- One token changes the future context.
- Future gates then change.
- Pass/fail is discontinuous.
- One seed per task adds noise.

EIGHT COMPLEMENTARY SCREENS

Association, paired model difference, and error-local timing answer different questions

REGRESSION

association · global
failures vs retained successes

REGRESSION

association · local
same contrast near first divergence

REGRESSION

paired · global
specialized text − base text

REGRESSION

paired · local
paired difference near divergence

IMPROVEMENT

association · global
improvements vs remaining base failures

IMPROVEMENT

association · local
same contrast near first divergence

IMPROVEMENT

paired · global
specialized text − base text

IMPROVEMENT

paired · local
paired difference near divergence

KNOWN FEATURES RE-APPEAR, BUT IN DIFFERENT CELLS

DSTK100 10168: rank #2 in improvement association (active fraction, E/V −7.67). CodeLlama 27: rank #45 in local paired regression.

New candidate 13147: rank #1 in global regression association (mean, E/V 5.24, support 95).

α=3 SWEEP: MANY DIRECTIONS MOVE THE SAME TASKS

A broad causal screen exposed task sensitivity—and the need for placebo directions

DSTK100 · 30 selected features

Baseline 15 / 80
25 / 30 features had positive net change
Best arm: 3048, +4 net
29 / 30 corrected task /1030

CodeLlama · 37 selected features

Baseline 0 / 50
11 / 37 produced one official pass
10 / 11 corrected the same task: /119
13147 alone corrected /490

INTERPRETATION

A pass transition is causal evidence that the residual perturbation mattered—but not yet that the selected latent was specific.

Repeated correction of the same tasks motivated orthogonal shams, matched latent controls, bidirectional steering, and dose curves.

TWO CANDIDATES, TWO SEMANTIC PROFILES

Natural TopK activations and CrossCoder geometry suggest different mechanisms

DSTK100 FEATURE 3048

TOP TOKENS
_ · test · Func · from · Task · import · def

CONTEXTS
helper functions · test blocks · imports
transitions into additional code units

GEOMETRY
decoder cosine base↔FT: 0.961
active in 1,427 / 2,608 texts

CODELLAMA FEATURE 13147

TOP TOKENS
indentation · newline · # · ... · TODO · pass · return

CONTEXTS
Hint: Use ... · placeholders
incomplete or scaffolded implementations

GEOMETRY
decoder cosine base↔merged: 0.275
active in 718 / 2,608 texts

3048 shared program-structure direction; semantically broad

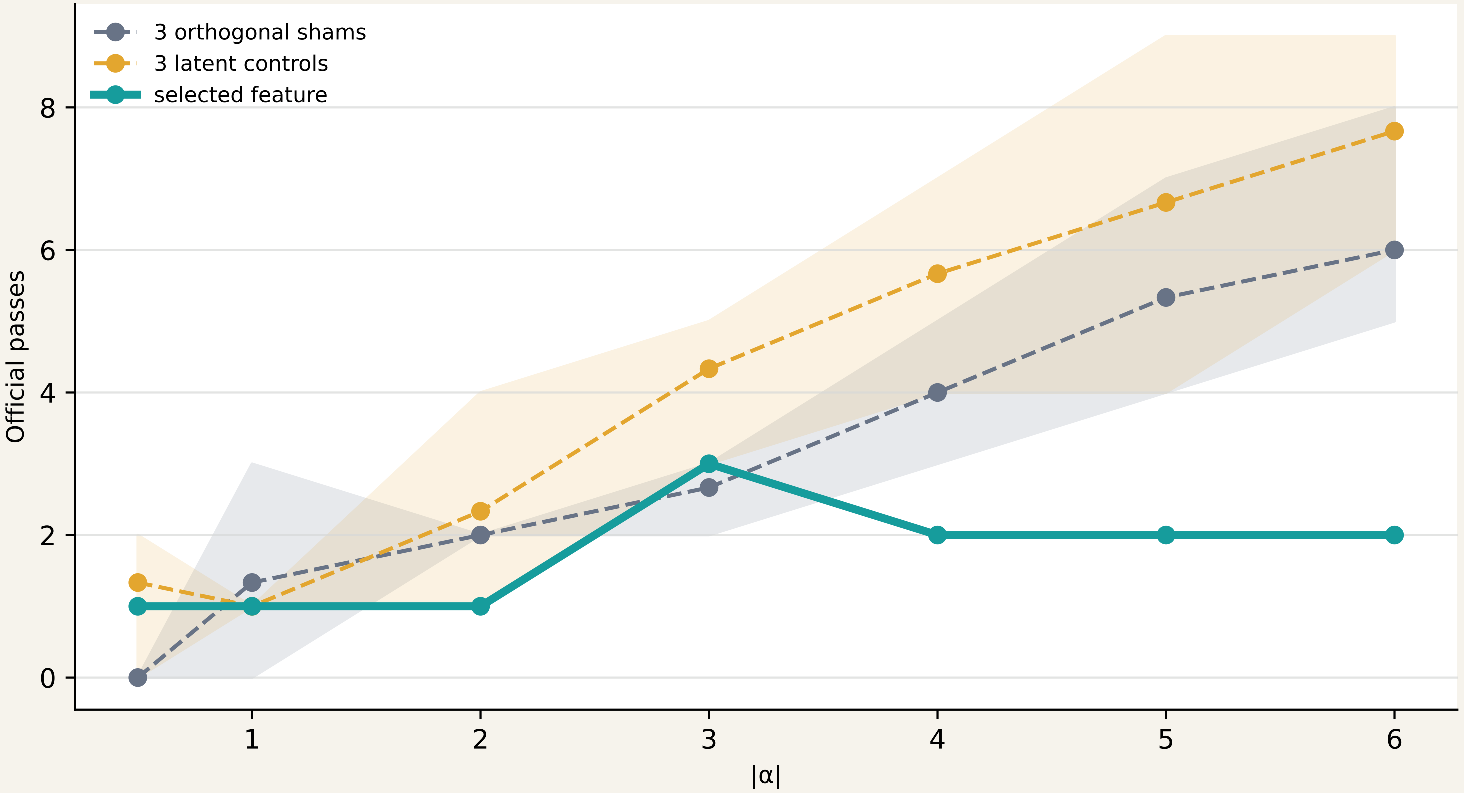
13147 model-differential incomplete-implementation direction

RECENT EXTENSION · SCREENING → SEMANTICS → CAUSAL STRESS TEST

FEATURE 3048: THE EFFECT IS NOT SPECIFIC

Seed 50000 · DeepSeek base · 80 selected contamination improvements

FAIL → PASS IN THE BASE MODEL



At |α|=3: 3048=3 passes · shams=2–3 · latent controls=3–5

The selected direction changes behavior, but matched and random perturbations are equally or more effective.

OUTPUTS CHANGED

3048: 13 → 37 / 80 across dose
At |α|=3: 27 changed
8 test-marker changes
4 import changes
8 other logic/text changes

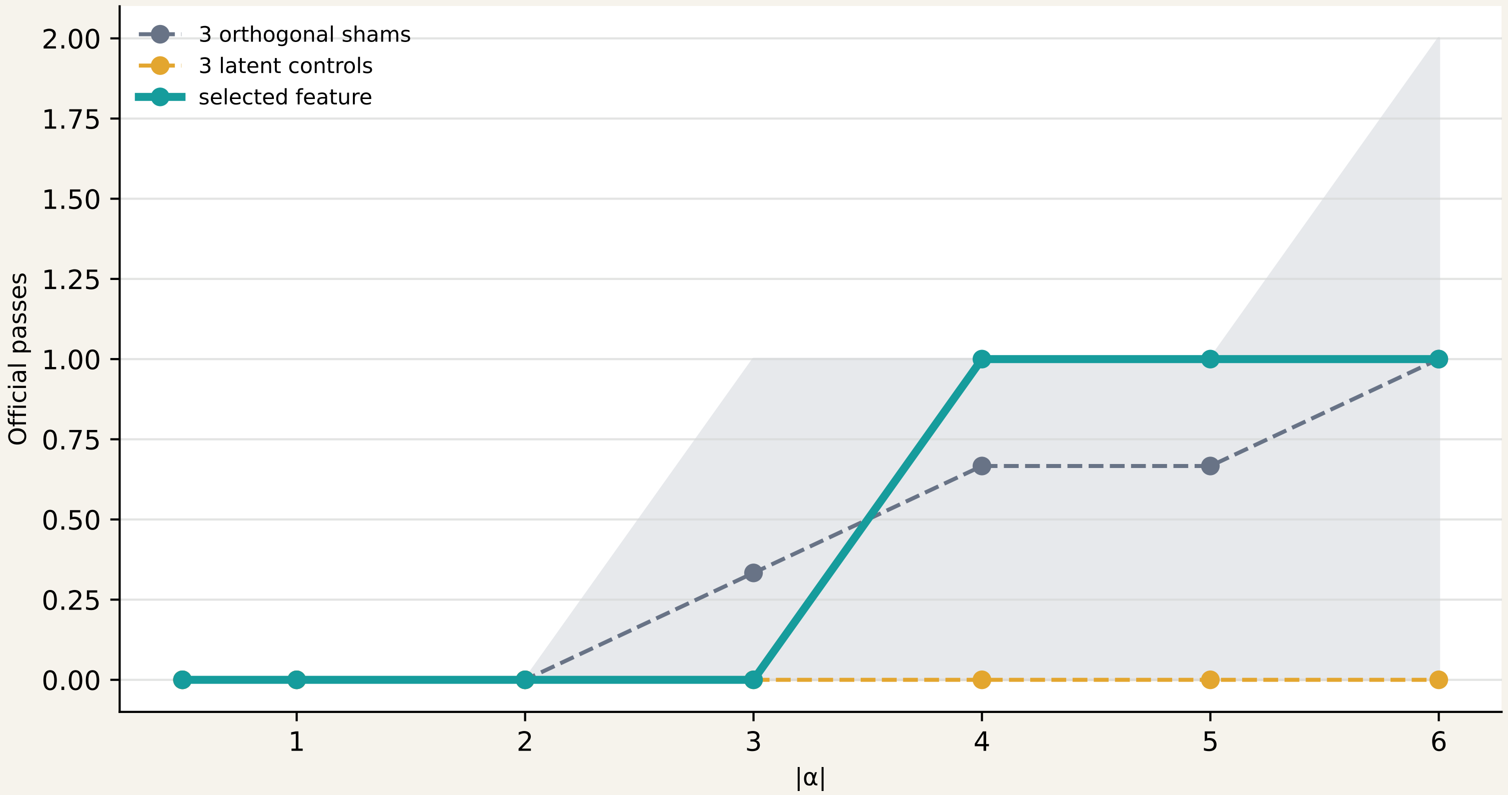
REVERSE DIRECTION

Adding 3048 to FT:
60 → 55 passes by +5/+6
6 → 40 outputs changed
No reverse sham control yet

FEATURE 13147: COHERENT, BUT PLACEBO-SENSITIVE

Seed 50000 · CodeLlama merged · 50 selected regressions

FAIL → PASS IN THE MERGED MODEL



Matched latent controls: 0 passes at every dose · Shams: up to 2 passes

The semantic match is strong, but one sham also repairs /490 at −6; specificity remains exploratory.

SEMANTIC CHANGE

13147 changes 8 → 35 / 50 outputs
Return-line changes: 3 → 18
At −4/−5/−6, /490 gains
parse → dump JSON → return result

REVERSE DIRECTION

Adding 13147 to base:
50 → 47 passes at +6
7 → 25 outputs changed
Losses emerge from +2

THE 116 ARMS ALTER MORE THAN PASS / FAIL

Exact extraction-v4 evaluated code reveals broad, dose-dependent trajectory changes

3048 FAMILY · 80 TASKS

SELECTED FEATURE, −0.5 → −6
changed outputs: 13 → 37
test-marker changes: 3 → 9
import changes: 4 → 9
other logic/text: 3 → 7

SHAMS AT −6
29–39 outputs changed · 5–8 passes

LATENT CONTROLS AT −6
42–47 changed · 6–9 passes

13147 FAMILY · 50 TASKS

SELECTED FEATURE, −0.5 → −6
changed outputs: 8 → 35
return changes: 3 → 18
other logic/text: 2 → 11

SHAMS AT −6
26–30 changed · return changes 14–18

LATENT CONTROLS AT −6
19–24 changed · 0 passes

RULE-BASED CHANGE TAXONOMY

test markers → imports → returns → function structure → comments → other logic/text (mutually exclusive, first matching category)

Steering effects are real and structured, but high-dose output change is not itself evidence of latent specificity.

RECENT EXTENSION · SCREENING → SEMANTICS → CAUSAL STRESS TEST

REVISED EVIDENCE LADDER

Controls separate trajectory control from feature-specific causal explanation

SUPPORTED

CrossCoder screens recover stable behavioral contrasts.

Selected directions causally alter code trajectories and official outcomes.

EXPLORATORY

13147 has a coherent meaning↔failure-mode link and a stable /490 repair across −4 to −6 in seed 50000.

NOT SUPPORTED

3048 does not outperform shams or latent controls.

A pass transition alone does not establish semantic specificity.

CRITICAL REPLICATION NOTE

The discovery α=−3 run used seed 1000+task_idx (1490 for /490). The focused sweep used input seed 50000.

Seed 1490 repaired /490 at −3; seed 50000 required −4 to −6. These are robustness runs—not exact replications.

NEXT CONFIRMATORY TEST

Re-run baseline, target feature, 3 shams, and 3 latent controls with the canonical seed convention and multiple paired seeds per α.

Feature 6404 remains the strongest controlled causal result in the project.